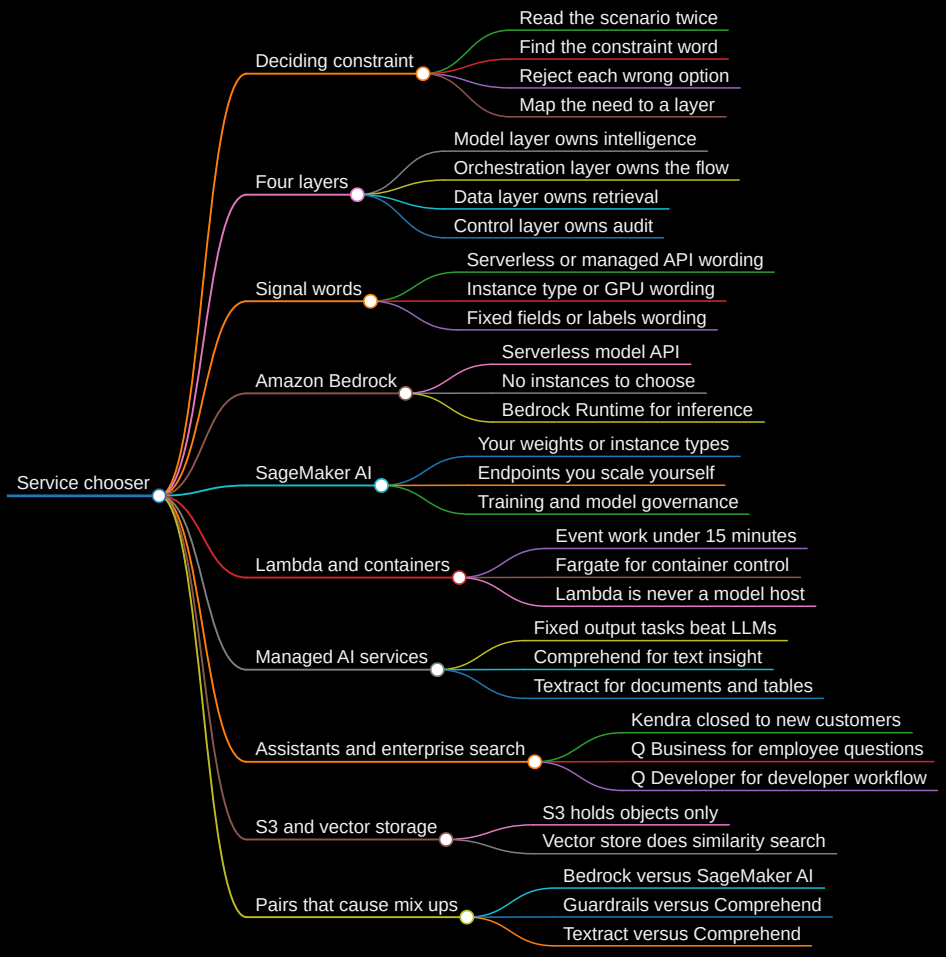
Choose the right service: Bedrock, SageMaker AI, Lambda, S3, and the managed AI services

After this module you can read a long scenario, find the one constraint that decides the service, and reject each other option with one reason. You will sort every service in scope into one of four layers. You will also know which signal words point to least operational overhead, which point to full control of instances, and when a managed AI service that is not generative beats a large language model.



The deciding constraint

Most long questions in this area hide the answer in one or two phrases. The rest of the stem adds detail that does not change the service. Start with the scenario. Ask what kind of output the system must produce, then follow the tree. Read the options only after the tree gives you a service family.

Exam tip: Write the deciding constraint in your own words before you look at the options. A useful form is "the system must ____ and must not ____."

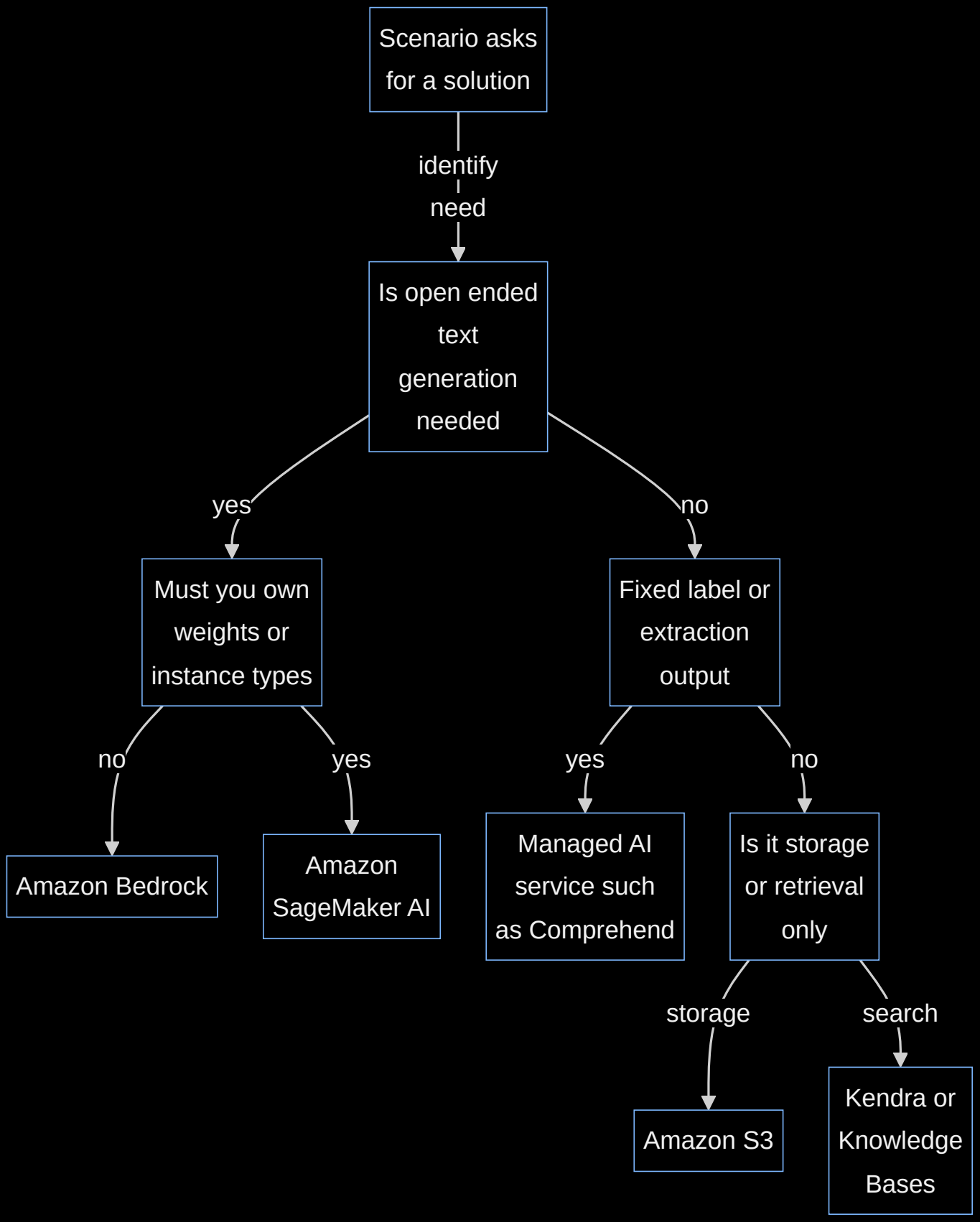


Figure: The first split is generation versus fixed output, then control versus managed.

Four layers and who owns each

Every AI system has four layers. The model layer produces intelligence, such as a generated answer or a prediction. The orchestration layer decides the order of calls,

retries, and branches. The data layer holds inputs, documents, and vectors. The control layer decides who may call what, what is logged, and what is safe to return.

Amazon Bedrock owns the model layer for foundation models. SageMaker AI owns the model layer when you bring model weights that you control. Managed AI services own the model layer for fixed tasks such as entity detection, text extraction, or translation. Each of these is a different owner of the same layer, so the layer alone does not decide the answer.

Lambda, Step Functions, and API Gateway own orchestration in most designs. Amazon S3 owns storage in the data layer. Knowledge Bases, vector stores, and Kendra own retrieval over that stored data. IAM, CloudWatch, CloudTrail, and Bedrock Guardrails sit in the control layer.

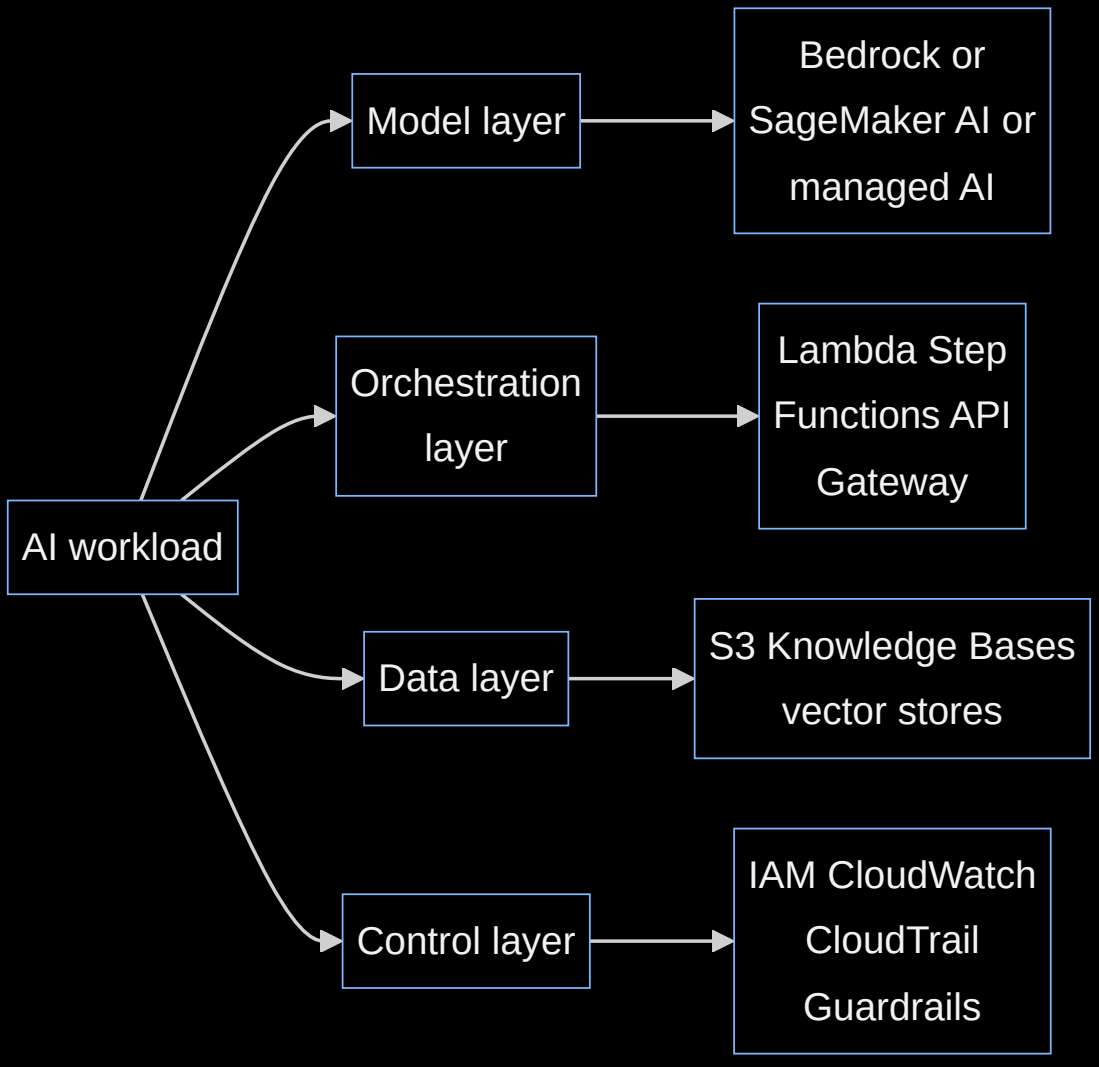


Figure: Each layer has an owner. Pick the owner of the layer the requirement touches.

Rule: Name the layer that the requirement touches before you name a service.

Trap: A control service in an option does not answer a model question. CloudWatch shows metrics and alarms. It does not generate text, and it does not host a model.

Signal words for the two big trade offs

Least operational overhead signals a managed or serverless choice. Phrases such as no servers to patch, pay only for requests, no capacity planning, or no model hosting point to Bedrock or to a managed AI service. Lambda also fits this signal when the work is short and event driven.

Full control of instances signals a service where you choose the hardware and the scaling rules. Phrases such as instance type, GPU model, custom container image, autoscaling policy, or runtime version point to SageMaker AI endpoints, ECS, EKS, or EC2. The stem may also ask for a specific Region, which narrows the choice further.

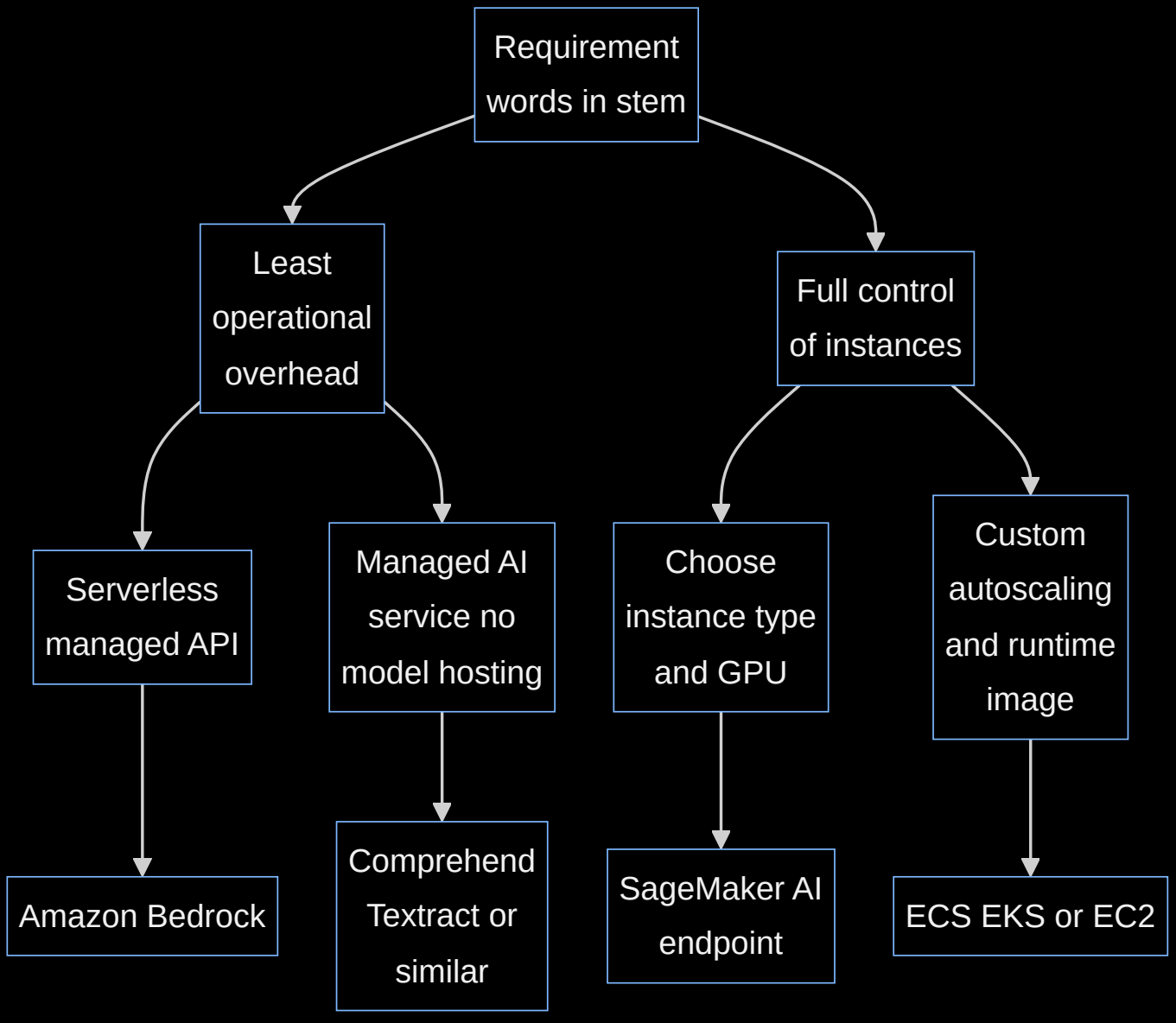


Figure: The two signal families lead to different owners. Both can lead to a managed choice.

Exam tip: Read the adjective before the noun. Least operational overhead and full control pull in opposite directions. When a stem asks for both, look for the managed option that still lets you choose the one thing that matters, such as the endpoint type or the model ID.

Trap: Full control is not a bonus feature. If the stem asks for least overhead, a self-managed fleet is wrong even when it would work.

Compute tree for custom code

Some stems describe custom code that must run, not a model call. Lambda fits short, event-driven code. The current Lambda limits allow up to 15 minutes for one invocation. Synchronous request and response payloads are capped at 6 MB each.

Fargate fits containers that run longer or need a custom image, and you do not manage the servers. EC2 and EKS give host-level control. The fetched sources do not give quota numbers for EC2 or EKS, so treat their limits as Not in source in this module.

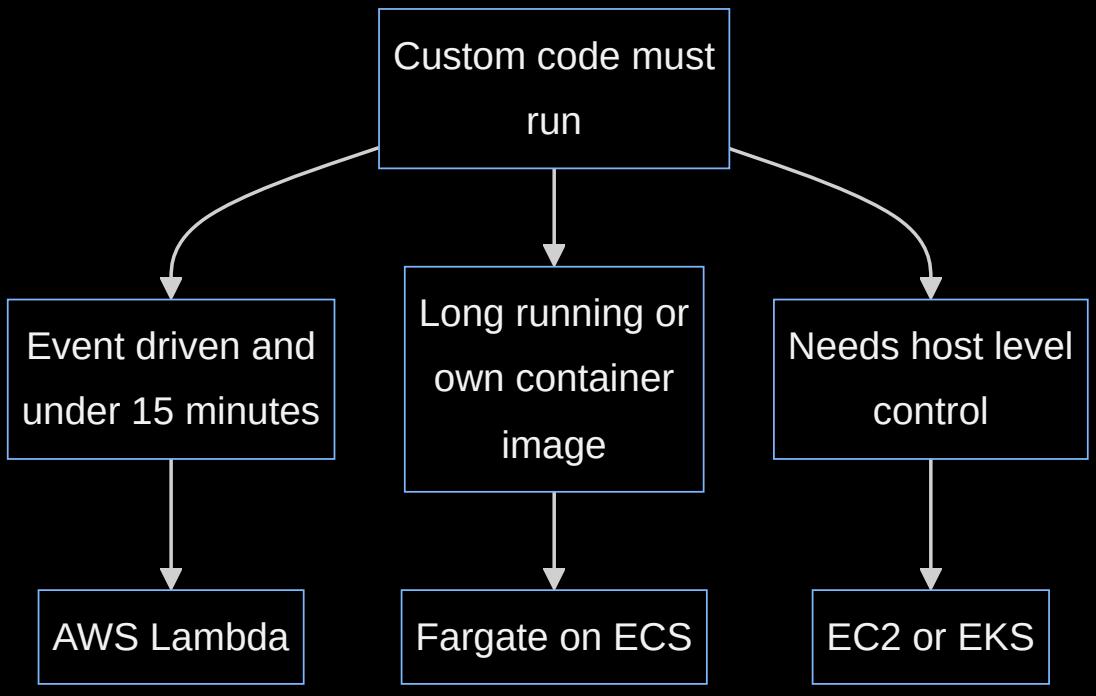


Figure: Compute choice depends on duration and how much host control the stem requires.

Source: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html and https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

---

Services in scope

This section covers each service once. Each entry gives the fact, the rule for choosing it, the trap for rejecting it, what it pairs with, and the limits that the sources confirm. Where a number is not in the sources, the entry says Not in source.

Amazon Bedrock

Fact: Bedrock is a managed service that gives API access to foundation models from several providers. The AWS overview says it supports more than 100 models at the time of the fetch. You call the models through a runtime endpoint, and you do not run model servers yourself.

Fact: The bedrock endpoint manages and deploys models. The bedrock-runtime endpoint runs inference, such as Converse, ConverseStream, InvokeModel, and InvokeModelWithResponseStream. The bedrock-agent-runtime endpoint runs agents and knowledge base retrieval. The course slides list these endpoint roles. The AWS overview recommends bedrock-runtime for new applications.

Rule: Choose Bedrock when the stem needs a foundation model, you do not own the weights, and you want little infrastructure work.

Rule: Choose Bedrock when one message format must work across several providers. The Converse operation takes one message structure for the models that support it. This matches stems that ask for one API across models.

Trap: Bedrock is wrong when the stem needs a custom open weight model on instances that you pick. Bedrock Custom Model Import can bring a model that you trained elsewhere into Bedrock. Check the stem for an import option before you reject Bedrock for a custom model.

Fact: Knowledge Bases take documents from S3 and need an embedding model. You can control the vector dimension and the chunking. A serverless OpenSearch instance can serve as the default vector store for development. An ingestion job, started with StartIngestionJob, syncs new or changed content. These details come from the course slides.

Fact: Provisioned Throughput is bought in Model Units or tokens. You must provision for customized models. You pass the provisioned model ARN as the modelId in the inference call. Cross-Region inference profiles exist for Bedrock endpoints, according to the course slides.

Fact: Bedrock batch inference takes prompts from S3 and writes responses back to S3. This fits large offline jobs where no user waits.

Bedrock pairs with Knowledge Bases for retrieval, Guardrails for model safety, Lambda for orchestration, S3 for batch files, and Provisioned Throughput for fixed capacity. The course slides also say that Bedrock Agents is deprecated for new customers and

that AWS recommends AgentCore. Check the current AWS documentation before you rely on that claim in a scenario.

Exam tip: When a stem names a model family and asks for the same call pattern across several models, the word that matters is the unified API, not the model name.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html. Course slide extract, pages 20, 22, 36, 48, 245, 290, 291, 317, and 318.

Amazon SageMaker AI

Fact: Amazon SageMaker AI is a fully managed machine learning service. You build, train, and deploy models into a hosted environment that you configure. AWS renamed Amazon SageMaker to Amazon SageMaker AI on December 3, 2024. The legacy API namespaces and console URLs did not change.

Fact: The course slides say JumpStart offers more than 150 open source models with one-click deployment. Endpoints can host models up to 500 GB, and the health check and download timeouts may need tuning for large models. Multi-model endpoints and inference components let one endpoint host several models with separate scaling policies.

Fact: The course slides give example instance choices. Large GPU models use ml.p4d.24xlarge. Small models such as named entity recognition use ml.c5.9xlarge on CPU. These are examples from the slides, not a full instance catalog.

Rule: Choose SageMaker AI when the stem names your own weights, an instance type, a custom container, or endpoint level autoscaling.

Rule: Choose SageMaker AI asynchronous inference when requests are large or slow and the client does not wait on the connection. Asynchronous inference appears in the course slides. Its numeric limits are Not in source for this module.

Trap: SageMaker AI is wrong when the stem asks for no hosting work and a managed foundation model meets the need. A hosted endpoint runs on instances that you pay for while they run. The sources in this module give no cost comparison, so cost claims are Not in source.

Fact: Your application formats the request body for a SageMaker endpoint. SageMaker does not reshape the input for you. The course slides make this point about structured data and LLM payloads.

Fact: Model Registry tracks model versions and their approval status. Model Monitor and Clarify can alert through CloudWatch when bias or drift appears. The course

slides cover both features.

SageMaker AI pairs with Model Registry, Pipelines, Model Monitor, Clarify, CloudWatch, API Gateway in front of endpoints, and Bedrock Custom Model Import when the team later wants serverless invocation.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html. Course slide extract, pages 78, 300, 316, 317, 318, 327, and 331.

AWS Lambda

Fact: Lambda is designed for short-lived compute tasks that do not keep state between invocations. The current limits allow up to 15 minutes for one invocation. A function can use 128 MB to 10,240 MB of memory, and CPU power scales with memory. At 1,769 MB, a function has the equivalent of one vCPU.

Fact: The synchronous invocation payload limit is 6 MB for the request and 6 MB for the response. Asynchronous invocations allow 1 MB. A streamed response can reach 200 MB, and the bandwidth limit applies after the first 6 MB of a streamed response.

Fact: A zip deployment package can be 50 MB zipped when uploaded through the API, and 250 MB unzipped. A container image can reach 10 GB uncompressed. The default concurrency is 1,000 executions per Region, and that quota can be raised.

Fact: API Gateway has a default throttle of 10,000 requests per second. The Lambda concurrency default can be lower than that. The AWS limits page recommends raising Lambda concurrency to match expected traffic.

Rule: Choose Lambda for glue code. Typical glue code validates input, redacts text, calls Bedrock, and shapes the response.

Rule: Choose Lambda response streaming when the stem needs streamed model output through a Lambda integration and the payload stays inside the streamed response limit.

Trap: Lambda is not a model host. The fetched limits page lists memory and CPU but no GPU option. When a stem asks for GPU inference, Lambda is wrong. The GPU claim is Not in source for Lambda in this module.

Trap: A job that runs longer than 15 minutes cannot stay on Lambda. Use a container service, a SageMaker AI asynchronous endpoint, or a workflow that splits the work into steps.

Trap: Lambda reserved concurrency does not raise a Bedrock quota. It only limits how many Lambda copies run at once. Bedrock throttling needs a Bedrock fix such as quota work, retries with backoff, or a queue.

Lambda pairs with API Gateway for synchronous APIs, SQS for buffered work, EventBridge for event routing, Step Functions for branching workflows, AppConfig for runtime settings, and CodeDeploy traffic shifting for alias updates. The course slides describe this last pairing for Lambda aliases.

Source: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html. Course slide extract, pages 354, 355, 356, 359, 364, and 388.

EC2, ECS, EKS, and Fargate

Fact: AWS Fargate runs containers with Amazon ECS without your managing servers or EC2 clusters. You package the application in a container, set the CPU and memory, define networking and IAM, and launch it. Each Fargate task has its own isolation boundary and does not share the kernel or network interface with another task.

Fact: Fargate Spot runs interruption tolerant tasks at a discount. AWS gives a two-minute warning before it reclaims the capacity. A load balancer target group for Fargate tasks must use the ip target type.

Rule: Choose Fargate on ECS when the stem wants containers, a custom runtime, and no server management.

Rule: Choose EC2 when the stem needs host or operating system control that Fargate does not give. The fetched sources do not give EC2 limits, so EC2 quotas are Not in source.

Trap: Fargate does not let you pick a host instance type. A stem that asks for a named GPU instance type is not a Fargate question.

Trap: EKS adds Kubernetes operations work. A stem that asks for the least operational overhead does not point to EKS unless it also requires Kubernetes.

Containers pair with an Application Load Balancer for HTTP traffic, Lambda or EventBridge for triggers, and Bedrock or SageMaker AI endpoints for model calls.

Source: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html. EC2 and EKS limits are Not in source.

Amazon Comprehend

Fact: Amazon Comprehend uses natural language processing to find entities, key phrases, language, sentiment, targeted sentiment, syntax, and personally identifiable information. Its pre-trained models need no training data from you.

Fact: Custom classification and custom entity recognition build custom models from your own labeled data with an AutoML process. Built-in features accept UTF-8 text. Custom classification and custom entity recognition also accept image, PDF, and Word files. Real-time analysis fits small workloads. Asynchronous jobs fit large document sets.

Fact: The DetectToxicContent operation takes up to 10 text strings. Each string can hold 1 KB, and the whole list can hold 10 KB. The API page says English is the only supported language at present, although its list of valid codes is longer. Check the current language note before you rely on a non-English toxicity check.

Fact: AWS will remove topic modeling, event detection, and prompt safety classification for new customers after April 30, 2026. Toxicity detection is not on that list in the fetched API page.

Fact: Comprehend may store your content to improve its pre-trained models. You can encrypt job output and the volume data with your own KMS key.

Rule: Choose Comprehend when the output must be a fixed label, a score, or a list of entities, and the stem wants a deterministic answer for code to act on.

Rule: Choose Comprehend DetectToxicContent before a model call when the stem asks for a toxicity score on English user text and a code path must stop the request.

Trap: Comprehend is wrong when the stem asks for a summary, a reply, or new prose. It returns labels and scores, not generated text.

Trap: Comprehend built-in analysis is wrong for scanned images. Extract the text with Textract first. The pre-trained features take UTF-8 text.

Trap: When the stem has a data retention or residency rule, read the storage statement before you accept Comprehend. The fetched page says Comprehend may store content.

Comprehend pairs with Lambda for preprocessing, Bedrock for generation after redaction, Textract before text analysis, and Guardrails for model level safety.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html and https://docs.aws.amazon.com/comprehend/latest/APIReference/API_DetectToxicContent.html. Course

slide extract, pages 57 and 118.

Amazon Textract

Fact: Amazon Textract detects typed and handwritten text and extracts text, forms, and tables from documents. The Analyze Document operation supports Queries, which extract specific values from a document. AnalyzeExpense handles invoices and receipts. AnalyzeID handles identity documents. Analyze Lending routes mortgage package pages to the right analysis.

Fact: Textract accepts image files and PDF files. Synchronous operations suit single-page documents. Asynchronous operations handle multipage documents. API quotas can be raised through Service Quotas. File size and language limits cannot be raised.

Rule: Choose Textract when the input is a scan, photo, or PDF and the output must be text, key values, or table cells.

Rule: Choose AnalyzeExpense when the stem names invoices or receipts, and AnalyzeID when it names identity documents.

Trap: Textract does not write summaries. Queries return values found on the page. A summary or a reasoned answer needs a model such as Bedrock after extraction.

Trap: Textract does not read audio. Speech input goes to Transcribe.

Textract pairs with Comprehend on the extracted text, Bedrock for a summary after extraction, S3 for document input, and Bedrock Data Automation where the stem asks for a newer structuring step.

Source: https://docs.aws.amazon.com/textract/latest/dg/what-is.html. Course slide extract, pages 79 and 118. Page size limits for synchronous calls are Not in source.

Amazon Transcribe

Fact: Amazon Transcribe converts speech to text. Custom vocabularies add brand names, acronyms, and technical words. Custom language models learn context from your own domain text. The course slides say that using both features gives the highest accuracy for domain speech.

Rule: Choose Transcribe when the input is recorded speech, such as call recordings or meetings, and the output must be a transcript.

Trap: Transcribe does not read text inside documents. Use Textract for that job.

Trap: Transcribe does not translate. Pass the transcript to Translate or to a model if the stem also asks for another language.

Transcribe pairs with Translate for multilingual captions, Comprehend for entity analysis of the transcript, and Bedrock for a summary of the transcript.

Source: course slide extract, pages 112 and 603. Transcribe quotas and supported formats are Not in source for this module.

Amazon Translate

Fact: Amazon Translate is a text translation service that works on demand. It translates unstructured text documents and lets applications work in several languages. It integrates with S3 document repositories, DynamoDB, Aurora, and Redshift, and it works from Lambda and AWS Glue.

Rule: Choose Translate when the stem asks for the same meaning in another language and the output must stay faithful to the source text.

Trap: Translate does not create new content. When the stem asks for a new draft in another language, a model such as Bedrock is the better fit.

Translate pairs with Transcribe for speech input, Polly for spoken output, and Comprehend for analysis after translation.

Source: https://docs.aws.amazon.com/translate/latest/dg/what-is.html. The supported language count and custom terminology options are Not in source for this module.

Amazon Polly

Fact: Amazon Polly converts text into lifelike speech. It offers generative, long-form, neural, and standard voices. You pay only for the text you synthesize, and you can cache and replay the speech. AWS states that Polly is certified for HIPAA and PCI DSS workloads.

Rule: Choose Polly when the output must be spoken audio from known text.

Trap: Polly does not understand or write text. If the stem asks for the words first, use a model and then send the text to Polly.

Polly pairs with Bedrock for generated scripts and with Translate for spoken output in another language.

Source: https://docs.aws.amazon.com/polly/latest/dg/what-is.html. Lexicon and speech markup options are Not in source for this module.

Amazon Rekognition

Fact: Amazon Rekognition analyzes images and video. The API can analyze files stored in S3. Features include object and scene detection, text in images, unsafe content detection, face analysis, celebrity recognition, and image properties.

Fact: Custom Labels trains a classifier on your own labeled images. The course slides say a few hundred images or fewer can be enough. Content moderation can use custom adapters, and a human review step can use Amazon Augmented AI.

Rule: Choose Rekognition when the input is an image or video and the output is a visual label, a moderation flag, or a face match.

Trap: Rekognition does not analyze the meaning of written text. For text moderation, use Comprehend toxicity or Guardrails. For text in documents, use Textract.

Rekognition pairs with S3 for image input, Lambda for event handling, and Augmented AI for human review of uncertain results.

Source: https://docs.aws.amazon.com/rekognition/latest/dg/what-is.html. Course slide extract, pages 598 and 599.

Amazon Kendra

Fact: Amazon Kendra is a managed intelligent search service. It uses semantic ranking, connects to data repositories through connectors, and filters results by user or group access to documents. Customers handle user authentication and authorization.

Fact: The Kendra GenAI index is built for retrieval augmented generation. Amazon Q Business and Amazon Bedrock Knowledge Bases can use it. Editions are GenAI Enterprise, Basic Enterprise, and Basic Developer. The Basic Developer edition is for proof of concept work, not production.

Fact: The Kendra documentation states that the service is no longer open to new customers. It recommends Amazon Bedrock Knowledge Bases for similar capability.

Rule: Choose Kendra when the stem names an existing Kendra index, or when it needs document level permission filtering across an enterprise repository and the availability note does not block the choice.

Trap: The course slides use Kendra in a legal question example. That slide is older than the availability note. Do not choose Kendra for a new build when the stem asks for a new deployment, because AWS directs new customers to Knowledge Bases.

Kendra pairs with Q Business as a retriever, with Bedrock for generation, and with Cognito for user context on the query. Index quotas are Not in source for this module.

Source: https://docs.aws.amazon.com/kendra/latest/dg/what-is-kendra.html. Course slide extract, pages 662 and 663.

Amazon Q Business

Fact: Amazon Q Business is a fully managed generative AI assistant for employees. It answers questions from enterprise data it can access, gives citations, and respects the permissions of the user. It supports connectors such as S3, SharePoint, and Salesforce, and it uses IAM Identity Center for user access.

Fact: The course slides say Q Business is built on Amazon Bedrock, but you cannot choose the underlying foundation model. The AWS page says the service is no longer open to new customers and points to Amazon Quick for similar capability.

Rule: Choose Q Business when the users are employees who ask questions of company content and the stem does not ask you to build an app.

Trap: Q Business is wrong for a customer-facing feature that needs your own prompts, your own model choice, or a runtime API inside your application. Use Bedrock for that.

Q Business pairs with Kendra or S3 as data sources and with IAM Identity Center for sign in.

Source: https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/what-is.html. Course slide extract, page 260.

Amazon Q Developer

Fact: Amazon Q Developer is a generative AI assistant that helps you understand, build, extend, and operate AWS applications. In an IDE it can chat about code, give inline completions, generate new code, scan code for security issues, and perform code upgrades. It is built on Amazon Bedrock.

Fact: Amazon Q Developer has a Free tier and a Pro subscription. AWS states that IDE plugins reach end of support on April 30, 2027, and it points to Kiro for similar capability.

Rule: Choose Q Developer when developers need help writing, reviewing, or upgrading code inside an IDE or the console.

Trap: Q Developer is not the model that serves your customers. Choose Bedrock for runtime model responses inside an application.

Trap: For a new rollout, read the IDE plugin end of support note before you choose Q Developer for an IDE workflow.

Q Developer pairs with IAM Identity Center for developer sign in and with the Bedrock service that the application itself uses at runtime.

Source: https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/what-is.html.

Bedrock Data Automation

Fact: The course slides describe Bedrock Data Automation as the newer way to turn unstructured content, such as PDF text, into structured output. It sits under the Bedrock family in the course.

Rule: Choose Bedrock Data Automation when the stem names it, or when it asks for a managed structuring step that stays inside Bedrock. Choose Textract when the stem asks for exact text, forms, or table cells from a scan.

Trap: Do not treat Bedrock Data Automation as a plain text reader. Its output options, quotas, and supported formats are Not in source for this module.

Source: course slide extract, page 79. No AWS page for this service was fetched for this module.

Amazon S3

Fact: Amazon S3 is object storage. In this module it is a storage service only. It holds files and does not run a model. The course slides place S3 in the data layer. It is the source for Knowledge Bases, the trigger path for ingestion, and the input and output store for Bedrock batch inference.

Fact: S3 Vectors stores vectors and offers vector search. It integrates with Bedrock Knowledge Bases according to the course slides. This is the one case where an S3 option can store vectors. Read the stem for the word vector before you assume S3 is only storage.

Rule: Choose S3 for source documents, batch input and output files, and object storage that other services read.

Trap: S3 does not redact text, does not search by meaning on its own, and does not call a model. An option that makes S3 the answer to a model or search requirement is wrong.

S3 pairs with Knowledge Bases, Bedrock batch inference, Textract and Rekognition as input stores, and vector stores for similarity search.

Source: course slide extract, pages 5, 36, 48, 148, and 290. S3 quotas are Not in source for this module.

X versus Y decisions for confusable pairs

Each pair below uses the same frame. First, a decision tree. Second, a three row comparison of purpose, input, and output or effect. Third, the one question to ask yourself. Last, the trap that the pair sets.

Bedrock versus SageMaker AI

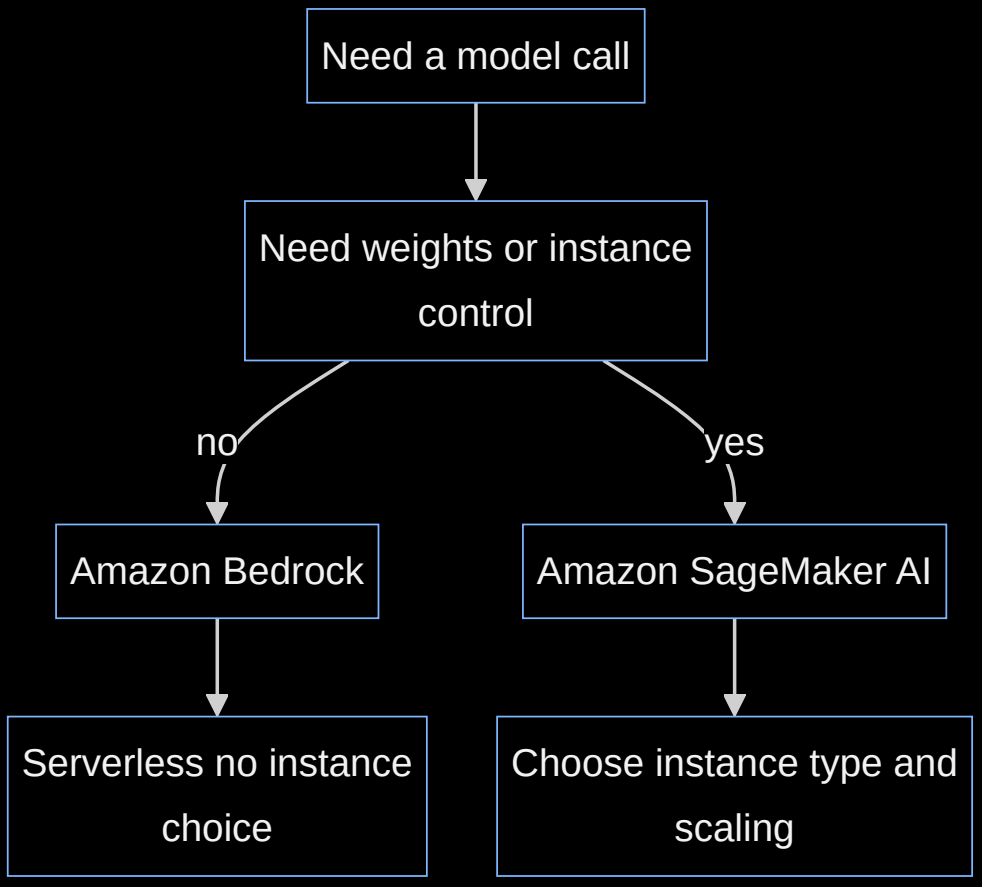


Figure: The model owner question splits these two services.

- Purpose: Bedrock gives API access to hosted foundation models. SageMaker AI builds, trains, and hosts models that you control.
- Input: Bedrock takes prompts or messages through Converse or InvokeModel. A SageMaker endpoint takes a payload in the format your container expects.
- Output or effect: Bedrock returns model output with no instance to size. SageMaker AI runs on instances you choose and scale.

Exam tip: The one question is whether the stem asks you to own the model weights or the instance type. If it does not, choose Bedrock.

Trap: Choosing SageMaker AI because it sounds more powerful. When the stem asks for minimal hosting work, the endpoint is the wrong answer even if it runs the model well.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html and https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html.

Bedrock versus Lambda

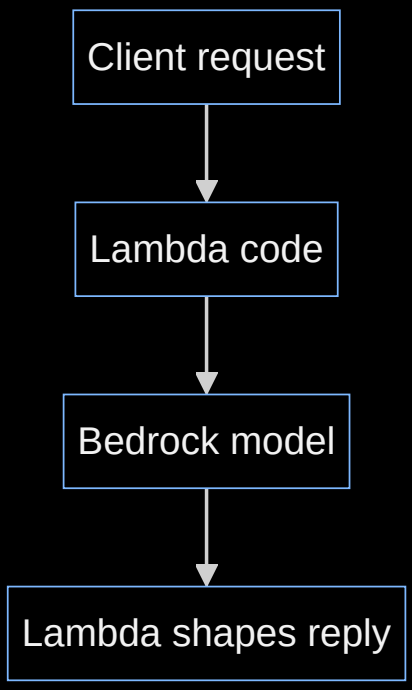


Figure: Lambda is usually the glue around the model call, not the model.

- Purpose: Bedrock runs inference. Lambda runs your code, which can call Bedrock.
- Input: Lambda receives events from API Gateway, S3, SQS, or EventBridge. Bedrock receives a model request.
- Output or effect: Lambda runs up to 15 minutes per invocation. Bedrock returns generated content.

Exam tip: Ask whether the option is naming the place where the model runs or the place where your glue code runs. Lambda rarely answers a question about which model service to use.

Trap: Lambda picked as a model host, or Bedrock picked as the scheduler, retry engine, or branching engine. Each service owns a different layer.

Source: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html.

Lambda versus Fargate, EC2, and EKS

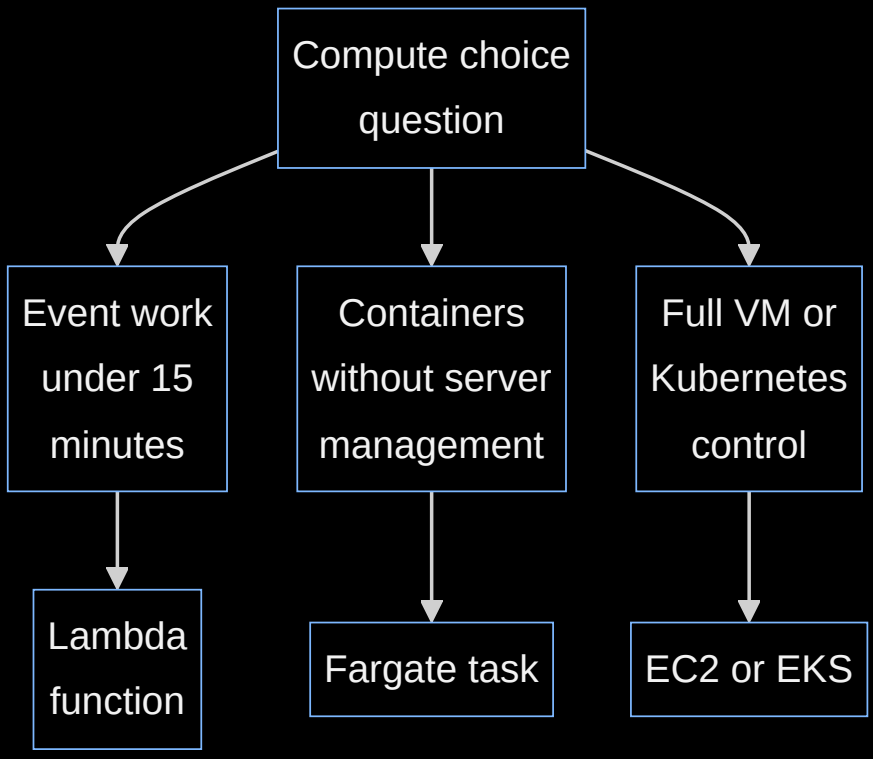


Figure: Duration and the amount of host control decide the compute service.

- Purpose: Lambda runs short functions on events. Fargate runs containers without servers. EC2 gives virtual machines. EKS runs Kubernetes.
- Input: Lambda takes event payloads, with a 6 MB synchronous limit. Fargate takes a container image and task settings.
- Output or effect: Lambda stops at 15 minutes per invocation. Fargate and EC2 suit long-running processes.

Exam tip: The one question is whether the work finishes inside 15 minutes and starts from an event. If it does, Lambda is a strong candidate. If not, compare containers.

Trap: EC2 or EKS chosen when the stem asks for least operational overhead and does not ask for host or Kubernetes control.

Source: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html and https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html.

Comprehend versus Bedrock for PII and toxicity

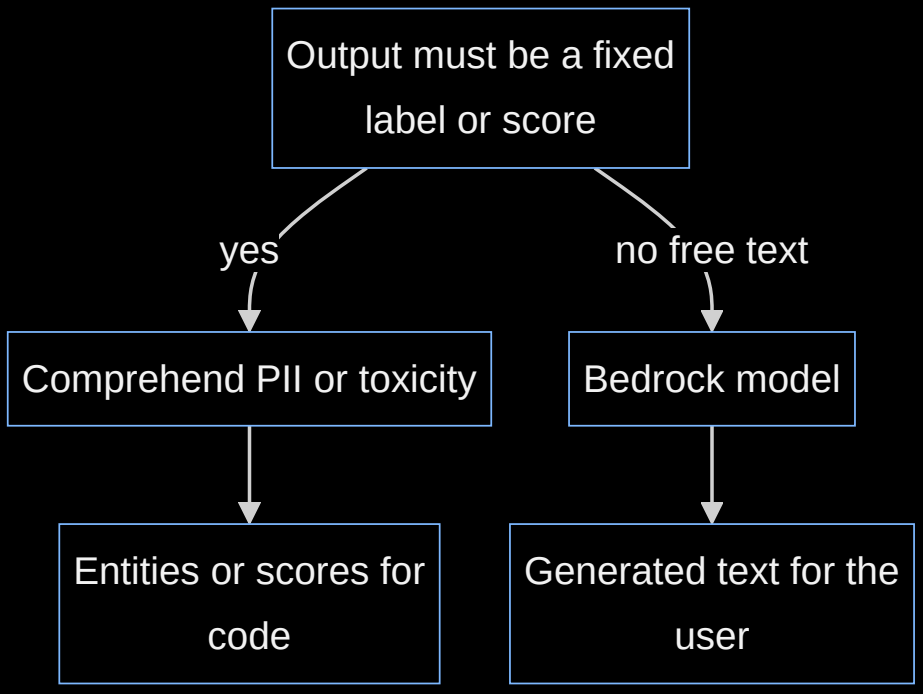


Figure: Fixed output points to a managed AI service. Free text points to a model.

- Purpose: Comprehend returns labeled insight, such as entities, PII types, or toxicity scores. Bedrock returns generated content.
- Input: Comprehend takes UTF-8 text in size limited segments. Bedrock takes a prompt or messages.
- Output or effect: Comprehend returns labels and scores that code can act on. Bedrock returns free text that needs its own checks.

Exam tip: The one question is whether the answer is a fixed label or a score. When it is, a non-generative service often beats an LLM.

Trap: Bedrock chosen to detect PII because it can read text. A stem that asks for a repeatable detection with a fixed result points to Comprehend. A model can miss items or change its output between calls.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html and https://docs.aws.amazon.com/comprehend/latest/APIReference/API_DetectToxicContent.html.

Comprehend versus Bedrock Guardrails

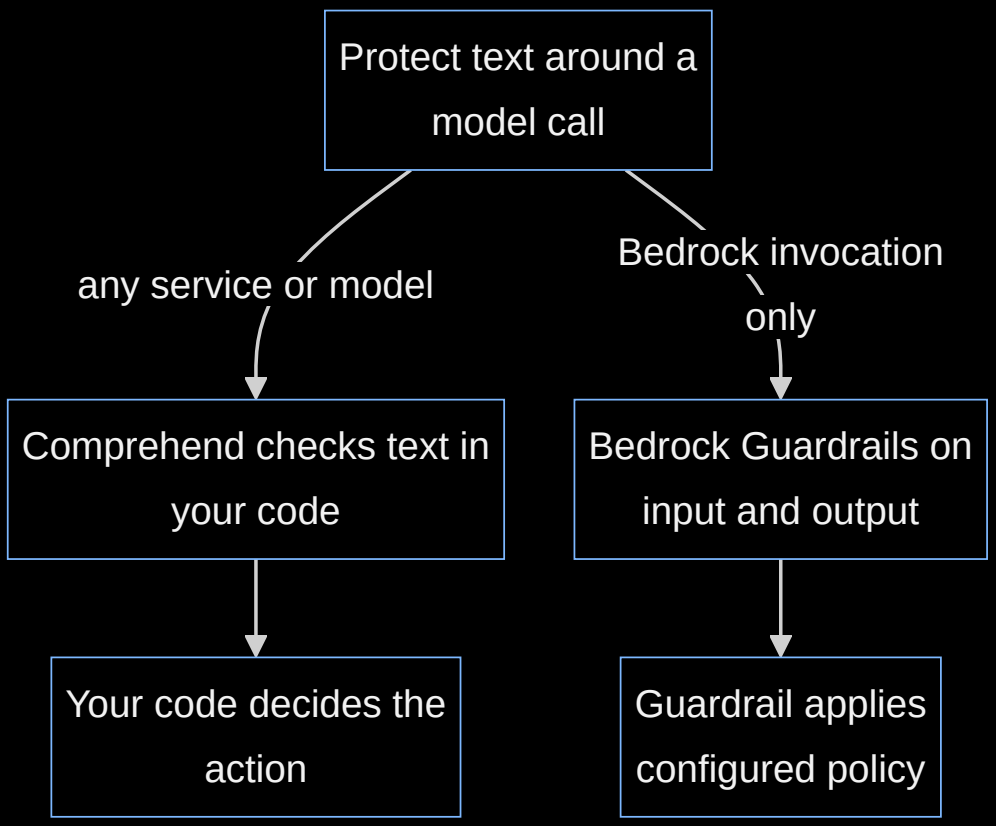


Figure: Comprehend is a check you place. Guardrails are a policy attached to a Bedrock call.

- Purpose: Comprehend returns detections and scores for text you send. Guardrails apply configured safety policies to a Bedrock call on the way in or out.
- Input: Comprehend takes text in your Lambda code. Guardrails take the prompt and the model response inside a Bedrock invocation.
- Output or effect: Comprehend gives labels for your code to act on. Guardrails block, mask, or flag content under the policy you set. The fetched sources do not list every Guardrails option, so policy detail is Not in source here.

Exam tip: The one question is whether the protection must sit before a call that is not Bedrock, or must sit on a Bedrock call itself. The first points to Comprehend. The second points to Guardrails.

Trap: Guardrails chosen alone when the stem asks for a redaction step before a non-Bedrock service. Comprehend chosen alone when the stem asks for model level safety on every Bedrock response.

Source: course slide extract, page 57. Bedrock Guardrails options are Not in source beyond the names in the course topic list on page 4.

Textract versus Comprehend

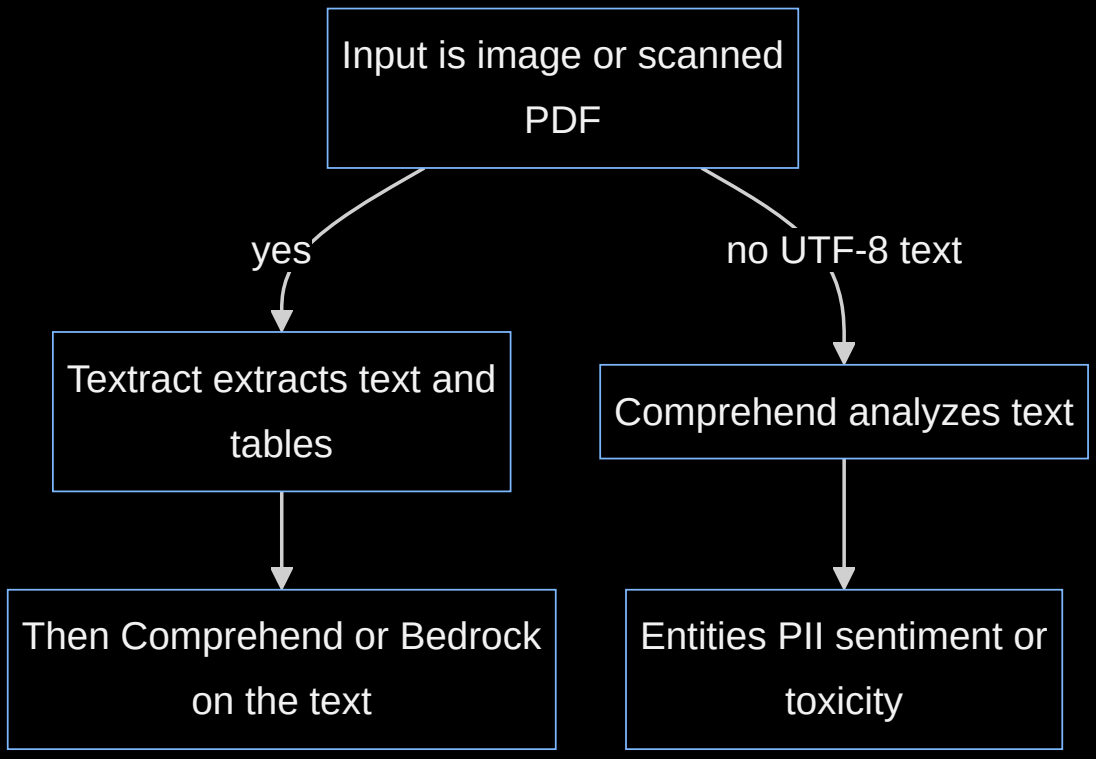


Figure: Textract reads the page. Comprehend reads the words that already exist as text.

- Purpose: Textract extracts text, forms, and tables from documents. Comprehend finds entities, PII, sentiment, and similar insights in text.
- Input: Textract takes image files and PDF files. Built-in Comprehend features take UTF-8 text.
- Output or effect: Textract returns text blocks, key values, and table cells. Comprehend returns labels and scores.

Exam tip: The one question is whether the input is still a picture or a PDF. If it is, extraction comes before analysis.

Trap: A scanned PDF sent straight to a built-in Comprehend feature. The built-in features take UTF-8 text, so the page must be extracted first.

Source: https://docs.aws.amazon.com/textract/latest/dg/what-is.html and https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html.

Kendra versus Bedrock Knowledge Bases

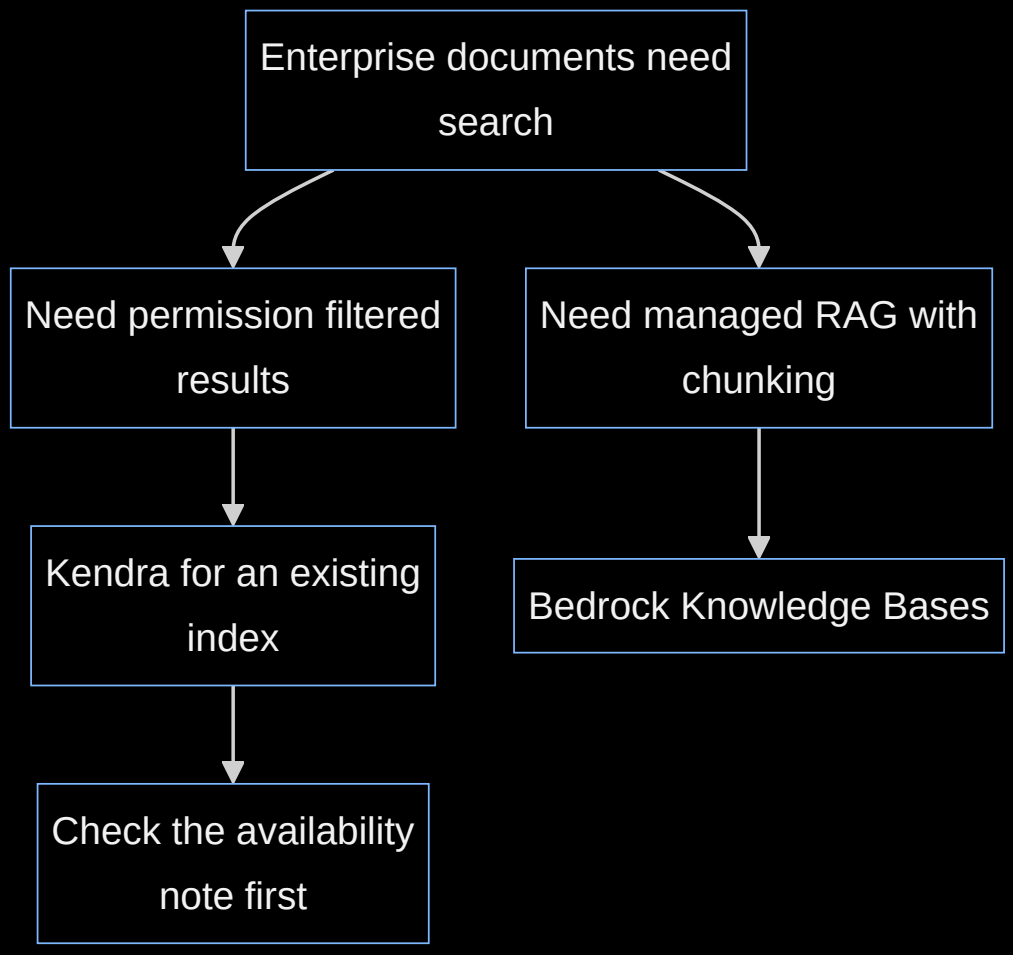


Figure: Both return retrieved content. Their availability and their owners differ.

- Purpose: Kendra is an enterprise search index with connectors and access filters. Knowledge Bases is a managed RAG workflow that chunks, embeds, and stores documents in a vector store.
- Input: Kendra takes documents from connectors and the identity of the user. Knowledge Bases takes documents from S3 and an embedding model.
- Output or effect: Kendra returns ranked results and answers. Knowledge Bases returns retrieved chunks, and it can generate an answer with a model.

Exam tip: The one question is whether the stem needs a search index with user level filtering, or a RAG pipeline you can build on Bedrock. For a new build, the availability note matters.

Trap: Kendra chosen for a new deployment because the stem mentions access control. AWS directs new customers to Knowledge Bases. Whether Knowledge Bases can enforce the same user filtering is Not in source for this module.

Source: https://docs.aws.amazon.com/kendra/latest/dg/what-is-kendra.html and https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html. Course slide extract, pages 36 and 662.

Amazon Q Business versus Bedrock

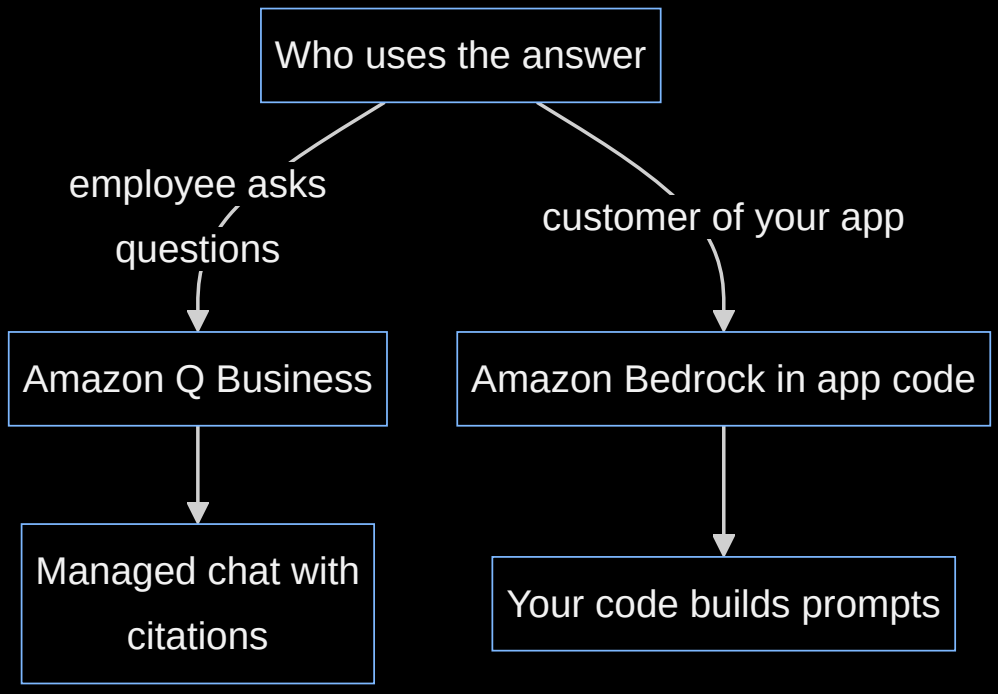


Figure: The user type decides the service. Q Business is an end product. Bedrock is a building block.

- Purpose: Q Business is a ready assistant for employees. Bedrock is an API for building model features into your own application.
- Input: Q Business takes employee questions and connected enterprise data. Bedrock takes prompts from your code.
- Output or effect: Q Business returns chat answers with citations. Bedrock returns model output that your code must present.

Exam tip: The one question is who the user is. An employee using a ready assistant points to Q Business. A customer of your product points to Bedrock.

Trap: Q Business chosen for a customer app that needs a model choice. The course slides say you cannot choose the foundation model in Q Business.

Source: https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/what-is.html and course slide extract, page 260.

Amazon Q Developer versus Bedrock

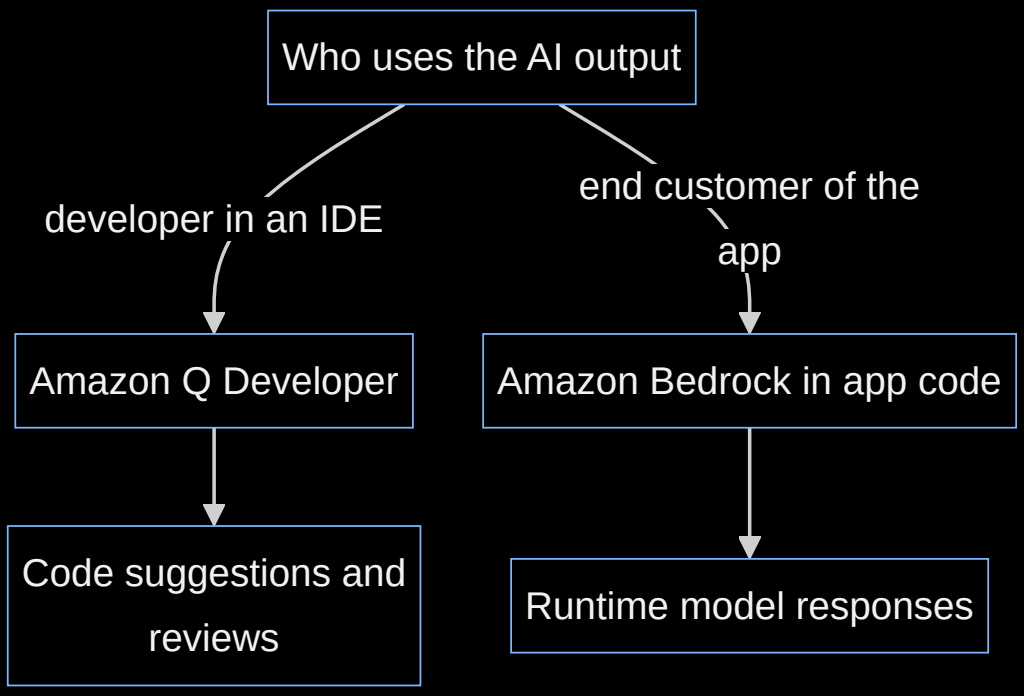


Figure: Q Developer changes how developers work. Bedrock changes what the application does.

- Purpose: Q Developer helps developers write, review, and upgrade code. Bedrock serves model responses inside an application.
- Input: Q Developer takes code and developer questions in an IDE or the console. Bedrock takes prompts from application code.
- Output or effect: Q Developer returns code, explanations, and security findings to the developer. Bedrock returns responses to the application.

Exam tip: The one question is whether the consumer of the output is a developer or a customer.

Trap: Q Developer chosen as the runtime model for customer chat. It serves developers. Bedrock serves the application.

Source: https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/what-is.html.

S3 versus a vector store

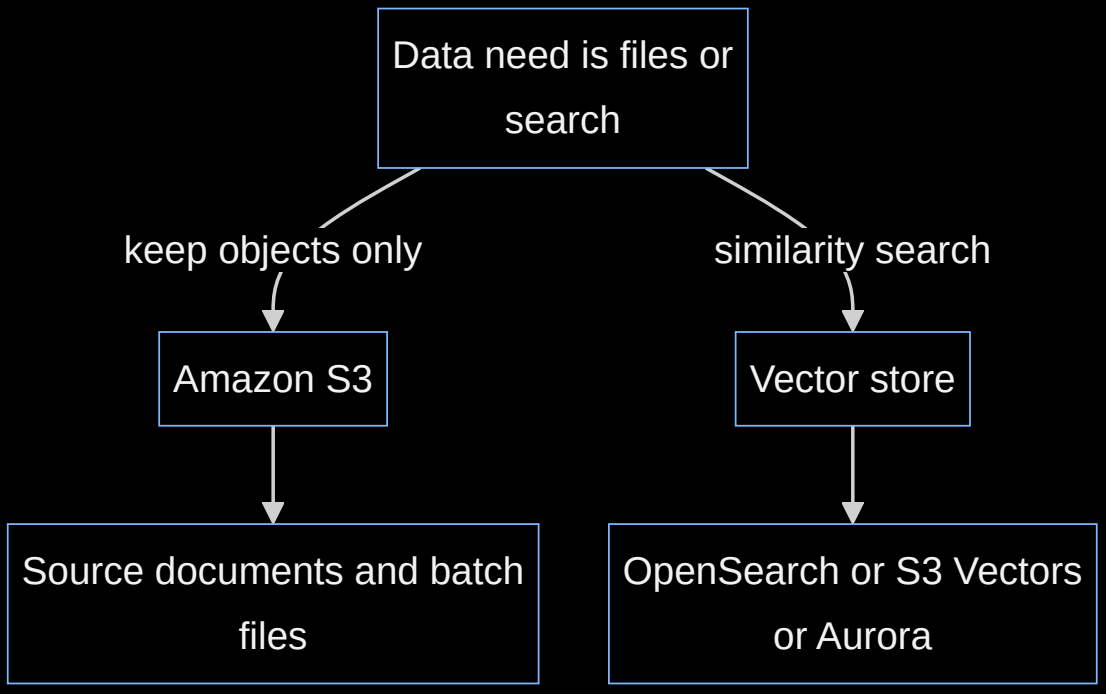


Figure: S3 keeps objects. A vector store finds nearby meanings.

- Purpose: S3 stores objects, such as documents, batch files, and images. A vector store stores embeddings and runs similarity search over them.
- Input: S3 accepts any object. A vector store accepts vectors with metadata.
- Output or effect: S3 returns objects by key. A vector store returns the nearest matches to a query vector.

Exam tip: The one question is whether the requirement says similar, meaning, or nearest. Those words point to a vector store, not to S3.

Trap: S3 chosen as the retrieval engine because the documents already live there. S3 keeps the files. Retrieval needs an index or a vector store.

Source: course slide extract, pages 36, 139, and 148. Vector store quotas are Not in source.

Read a long question

Long stems add detail that looks important. Use three passes. First, write the one sentence that says what the system must do. Second, mark the constraint words, such as operational overhead, instance control, Region, PII, scanned, or batch. Third, reject each wrong option with one reason, and keep the option that survives.

Exam tip: Mark only the words that change the service. Names of teams, company types, and dates usually do not change the answer.

Worked scenario one: drafting letters without staff to run servers

A lender wants a model to draft plain language letters for applicants. The team has no staff to patch servers and wants to pay only for use. Applicant names must not reach the model in plain text.

Constraint words: **no staff to patch servers**, **pay only for use**, and **must not reach the model in plain text**. The first two point to a serverless model API. The third adds a detection step before the model call.

- Self-managed GPU endpoint on instances: rejected, because the team wants no servers to patch and no instance work.
- Textract to read the names: rejected, because the input is text and Textract reads images and PDFs.
- Q Business as the drafting tool: rejected, because it serves employees and the letters go to applicants.
- Names stored in S3 and read by the model: rejected, because S3 only stores objects and does not hide names.

Rule: Answer is Amazon Bedrock for the drafting call, with a Comprehend PII step in Lambda before the call. Guardrails can add model side checks.

Worked scenario two: a fixed GPU type and custom scaling

A research group fine-tunes an open weight model. It must run on a fixed GPU instance type with custom scaling rules. The group also wants each model version to carry an approval status before deployment.

Constraint words: **fixed GPU instance type**, **custom scaling rules**, and **approval status**. The first two signal full control of instances. The third points to a model registry.

- Bedrock on demand: rejected, because the stem needs an instance type and Bedrock does not expose one.
- Lambda running the model: rejected, because Lambda is not a GPU model host and it stops at 15 minutes.
- Fargate with a GPU image: rejected, because Fargate hides the host, so you cannot pick the GPU instance type.
- Comprehend custom classification: rejected, because the task is a fine-tuned generative model, not a classifier over text.

Rule: Answer is a SageMaker AI endpoint with Model Registry approval status.

Worked scenario three: scanned intake forms in one Region

A clinic scans intake forms as PDF files. It needs a short summary of allergies and medications, and the summary must use only values found on the page. Patient data must stay in the same Region.

Constraint words: **scanned PDF**, **only values found on the page**, and **same Region**. The first means extraction comes first. The second means the values come from extraction, not from a model guess. The third rules out routing that moves requests to other Regions.

- Comprehend alone: rejected, because a scanned PDF needs extraction and Comprehend cannot write the summary.
- Kendra search: rejected, because the task is extraction from one form, not search across a repository.
- Cross-Region inference profile: rejected, because it routes requests to other Regions by design. The exact routing rules are Not in source for this module.
- Q Developer: rejected, because it serves developers, not clinic staff.

Rule: Answer is Textract for the values in the same Region, then Bedrock in that Region for the summary, with the summary limited to extracted fields.

Distractor patterns seen in practice items

Wrong options follow a small set of patterns. The patterns repeat across many topics, so learn the pattern as well as the service. Each pattern below names what the option gets wrong.

- **Right family, wrong layer.** A control service such as CloudWatch or CloudTrail appears where a model service is needed. It records or alarms on activity. It does not produce the answer.
- **Compute as model host.** Lambda or a container is offered as the place where a foundation model runs. These services run your code. The model is still a Bedrock or SageMaker AI endpoint.
- **Storage as intelligence.** S3 is offered as the place that searches, redacts, or answers. S3 holds objects. It needs a service on top.
- **Generative tool for a fixed task.** An LLM is offered for entity detection, PII detection, or text extraction. A fixed task has a managed service with a fixed output, which is easier to test.

- **Managed fix that adds hosting.** SageMaker AI or EKS is offered for a stem that wants least operational overhead. The option works, but it adds the work that the stem removes.
- **Assistant for the wrong user.** Q Business or Q Developer is offered for a customer app. These serve employees or developers. Bedrock serves the application.
- **Closed or old service.** Kendra or Q Business is offered for a new build. Check the availability note in the sources before you accept the option.
- **Synchronous tool for a long job.** Lambda is offered for work that needs more than 15 minutes. The fix is a container, an asynchronous endpoint, or a workflow.
- **Quota fix for a design problem.** Raising a quota or adding retries is offered when the stem needs a different service or a fixed capacity purchase. A quota fix does not change the owner of a layer.

Trap: A distractor that is true about the service but does not answer the constraint is still wrong. Reject it with the constraint in one sentence.

Drill

Read each prompt, answer in your head, then check the next line.

Prompt 1: A team needs a hosted foundation model with no servers to run and no weights of its own.

Rule: Amazon Bedrock.

Prompt 2: A team owns fine-tuned weights and must pick the GPU instance type and the scaling rules.

Rule: SageMaker AI endpoint.

Prompt 3: A job runs for forty minutes on a schedule and needs a custom container.

Rule: Fargate on ECS, not Lambda, because the job exceeds 15 minutes.

Prompt 4: Names and account numbers must leave the text before a model call, and the stem wants a managed detector.

Rule: Comprehend PII detection in Lambda before the Bedrock call.

Prompt 5: A scanned invoice must yield its line items as table cells.

Rule: Textract, with AnalyzeExpense for invoices.

Prompt 6: Employees ask questions about company policy and expect cited answers, with no app to build.

Rule: Amazon Q Business, after checking the availability note.

Prompt 7: Developers in an IDE want code suggestions and a security scan.

Rule: Amazon Q Developer.

Prompt 8: A set of stored photos must be checked for unsafe imagery.

Rule: Amazon Rekognition content moderation.

Prompt 9: Application text must become spoken audio for a voice feature.

Rule: Amazon Polly.

Prompt 10: A retrieval layer must find passages by meaning, and S3 alone cannot do it.

Rule: A vector store such as OpenSearch Service or S3 Vectors.

Prompt 11: A Bedrock call must refuse a forbidden topic in both the prompt and the response.

Rule: Bedrock Guardrails topic policy on the call.

Prompt 12: Learner text in English must get a toxicity score before a model call, and code must stop the request above a limit.

Rule: Comprehend DetectToxicContent in Lambda before the Bedrock call.

Sources

AWS documentation fetched for this module:

- https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html
- https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html
- https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html
- https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html
- https://docs.aws.amazon.com/comprehend/latest/APIReference/API_DetectToxicContent.h
- https://docs.aws.amazon.com/textract/latest/dg/what-is.html

- https://docs.aws.amazon.com/translate/latest/dg/what-is.html
- https://docs.aws.amazon.com/polly/latest/dg/what-is.html
- https://docs.aws.amazon.com/rekognition/latest/dg/what-is.html
- https://docs.aws.amazon.com/kendra/latest/dg/what-is-kendra.html
- https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/what-is.html
- https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/what-is.html

Course slide extract, cited by page number in each section. Items marked Not in source were not confirmed by the fetched pages.

All modules Next